手写小GPT学习笔记 | Day15 Transformer Block堆叠

Day15核心任务：将Day14验证通过的**单Transformer Block**进行多层堆叠，搭建GPT专属的**解码器骨架**，掌握深度堆叠的设计逻辑、维度守恒规则，为后续嵌入层、生成任务筑牢结构基础。

一、Day15 学习目标

- **理解堆叠逻辑**：搞懂GPT解码器“重复堆叠Block”的设计思路，深层网络的信息传递机制
- **实现多层堆叠**：封装可配置层数的GPT解码器类，支持自由设定Block数量
- **验证维度一致性**：确保堆叠后输入输出维度不变，无报错、无信息畸变
- **衔接后续模块**：预留嵌入层、位置编码、输出头的接口，保证结构兼容
- **排查深层隐患**：规避堆叠后梯度消失、维度错乱、掩码失效等问题

二、前置知识回顾

🔁 **单Block复盘**：单个Transformer Block = 多头注意力 + Pre-LN + 残差 + FFN，是GPT的“基础单元”

1. 为什么要堆叠Block?

- **分层提取特征**：浅层Block捕捉局部语法、词汇依赖，深层Block挖掘全局语义、逻辑关系
- **提升表达能力**：单层Block拟合能力有限，多层堆叠才能学习复杂的文本上下文依赖
- **GPT架构惯例**：原生GPT-2采用12层、GPT-3采用96层Block堆叠，小Demo用2-4层即可验证效果

2. 堆叠核心原则

- **维度严格守恒**：每一层Block的输入shape = 输出shape，全程保持 `[batch, seq_len, d_model]`
- **掩码全程复用**：因果掩码+填充掩码只需生成一次，所有Block共享同一套掩码
- **超参数统一**：所有Block的d_model、num_heads、dropout等超参数完全一致

三、核心代码实现（堆叠版GPT解码器）

基于Day14的单Block代码，封装**多层堆叠类**，代码可直接运行，注释标注堆叠关键点，兼容前期代码风格。

Day15 核心代码：Transformer Block堆叠

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import os

os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1"

# ===================== 复用Day14的基础组件（无需修改，直接调用）=====================
# 1. 掩码工具
def create_padding_mask(seq, pad_idx=0):
    mask = (seq == pad_idx)
    return mask.unsqueeze(1).unsqueeze(2)

def create_causal_mask(seq_len):
    mask = torch.tril(torch.ones(seq_len, seq_len, dtype=torch.bool))
    mask = ~mask
    return mask.unsqueeze(0).unsqueeze(0)

# 2. 缩放点积注意力
class ScaledDotProductAttention(nn.Module):
    def __init__(self, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)
    def forward(self, q, k, v, mask=None):
        d_k = q.size(-1)
        scores = torch.matmul(q, k.transpose(-2, -1)) / torch.sqrt(torch.tensor(d_k, dtype=torch.float32))
        if mask is not None:
            scores = scores.masked_fill(mask == True, -1e9)
        attn = F.softmax(scores, dim=-1)
        attn = self.dropout(attn)
        return torch.matmul(attn, v), attn

# 3. 多头注意力
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_model = d_model
```

```python
        self.num_heads = num_heads
        self.d_k = d_model // num_heads
        self.w_q = nn.Linear(d_model, d_model)
        self.w_k = nn.Linear(d_model, d_model)
        self.w_v = nn.Linear(d_model, d_model)
        self.w_o = nn.Linear(d_model, d_model)
        self.attn = ScaledDotProductAttention(dropout)
        self.dropout = nn.Dropout(dropout)

    def split_heads(self, x, batch_size):
        return x.view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)

    def combine_heads(self, x, batch_size):
        return x.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)

    def forward(self, q, k, v, mask=None):
        batch_size = q.size(0)
        q_proj = self.split_heads(self.w_q(q), batch_size)
        k_proj = self.split_heads(self.w_k(k), batch_size)
        v_proj = self.split_heads(self.w_v(v), batch_size)
        attn_out, _ = self.attn(q_proj, k_proj, v_proj, mask)
        out = self.combine_heads(attn_out, batch_size)
        return self.dropout(self.w_o(out))

# 4. FFN前馈网络
class FeedForwardNetwork(nn.Module):
    def __init__(self, d_model, dropout=0.1, expansion=4):
        super().__init__()
        self.linear1 = nn.Linear(d_model, d_model*expansion)
        self.linear2 = nn.Linear(d_model*expansion, d_model)
        self.gelu = nn.GELU()
        self.dropout = nn.Dropout(dropout)
    def forward(self, x):
        return self.linear2(self.dropout(self.gelu(self.linear1(x))))

# 5. 单Transformer Block
class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()
        self.attn = MultiHeadAttention(d_model, num_heads, dropout)
        self.ffn = FeedForwardNetwork(d_model, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
```

四、堆叠关键细节解析

1. ModuleList 为什么不能用普通List?

- **普通List**：不会被PyTorch识别为模型参数，训练时无法更新权重，导致模型不收敛
- **nn.ModuleList**：专门用于封装子模块，会自动注册所有Block的参数，支持训练、保存加载

2. 最终层归一化（Final Norm）的作用

多层堆叠后，特征分布会逐渐偏移，加一层**最终LayerNorm**可以收敛特征分布，让后续输出层（未实现）更易学习，是GPT、BERT等模型的标准设计。

3. 层数设置建议

- Demo测试：2-4层（速度快，无报错，适合验证结构）
- 小批量训练：6-8层（兼顾效果与速度）
- 严禁盲目堆层：层数过多+无残差保护会引发梯度消失，前期学习固定2-3层即可

五、Day15 避坑清单

💡 **堆叠高频错误**

- **错用List封装**：必须用nn.ModuleList，否则参数不生效
- **掩码重复生成**：掩码只需生成一次，所有Block共享，无需每层新建
- **超参数不统一**：不同Block的d_model、num_heads必须一致，否则维度报错
- **缺失Final Norm**：深层输出特征波动大，后续训练不稳定
- **层数过多**：新手阶段建议≤4层，避免梯度消失、训练变慢

六、今日成果验收+明日预习

今日达标标准

- ✅ 运行测试脚本无报错，输入输出维度完全一致
- ✅ 理解ModuleList堆叠原理，能自由调整Block层数

- ✅ 掌握掩码复用、维度守恒、Final Norm核心知识点

明日（Day16）预习

为堆叠好的解码器搭配**词嵌入层+位置编码**（sin/cos编码、可学习编码），让模型真正读懂文本序列，补齐GPT输入环节。

🎯 恭喜完成Day15学习！GPT骨架已搭建完成，距离完整模型只剩嵌入层和输出头，继续推进～

（注：文档部分内容可能由 AI 生成）